

onod: A Full-Rust CPU-Only Hybrid Search Engine with ONNX Transformer Embeddings and Sparse Fusion for 100% Multilingual Recall at 54 ms

Yudono Putro

Independent Research, Jakarta, Indonesia

September 2026

Abstract—We present **onod**, a full-Rust hybrid search engine that runs entirely on CPU with no GPU dependency. Dense retrieval uses an ONNX multilingual Transformer (384-dim, int8-quantized, Unigram 250k tokenizer, 128-token truncation) via the *ort* crate: *last_hidden_state* with attention-mask mean-pooling and L2 normalization, one vector per 600-char chunk. Sparse retrieval uses TF-IDF over a 30k vocabulary. Both signals are fused with Reciprocal Rank Fusion (0.6 dense / 0.4 sparse, $k=60$), financial-term boosting, character-trigram overlap on top-200 candidates, and cosine reranking. On 3 financial PDFs (4.3 MB, 3750 chunks, Apple M2 8-core, 16 GB) **onod** achieves **20/20 queries = 100% recall** — including English, Indonesian, and Chinese queries — at **54.0 ms average latency** (45.4–79.7 ms, target <100 ms). Indexing takes ~100 s on CPU due to serial per-chunk inference; we analyze this bottleneck and propose batch inference. If *model.onnx* is absent the system falls back to hash char n-gram embeddings without crashing.

Index Terms—hybrid retrieval, ONNX Runtime, multilingual embeddings, RRF fusion, Rust, CPU-only.

1 Introduction

Dense RAG pipelines encode every chunk with models such as BERT, MiniLM, LaBSE, BGE-M3, or multilingual-E5 [3]–[7]. For gigabyte-scale multilingual corpora this means millions of forward passes at ingestion, often exceeding 60 s on CPU and requiring GPU infrastructure. Dense models also underperform on exact matches (e.g. 62.714.280) due to embedding noise, while pure BM25 [1] misses cross-lingual paraphrases (*revenue* vs *pendapatan* vs Chinese queries).

onod compiles documents into **two complementary signals**: (i) dense Transformer embeddings executed on CPU via ONNX Runtime [11] with int8 quantization, and (ii) sparse TF-IDF keywords. Fusion is scale-free via RRF [2], plus financial boosting, trigram overlap, and a cheap cosine rerank — no GPU, no Python serving, no per-language indexes.

Contributions: (1) a **CPU-only ONNX dense path** (3 inputs, mean-pool, L2-norm, query/doc consistency with graceful hash fallback); (2) a **hybrid dense+sparse+RRF+boost+rerank** pipeline in Rust; (3) a **20-query trilingual benchmark** with 100% recall at 54 ms; (4) an analysis of the CPU indexing bottleneck (~100 s / 3750 chunks) with a batching roadmap.

2 System Architecture

Documents (PDF via *pdf_oxide*, text/MD/CSV/HTML directly) are NFKC-normalized (Arabic diacritics stripped, whitespace collapsed), split into sentences (including CJK terminators), then packed into ~600-char chunks with 140-char overlap. Tables longer than 900 chars are hard-split at word boundaries. Headings (ALL-CAPS, BAB/Chapter) are kept as metadata.

2.1 Dense path (ONNX Transformer)

Tokenizer: Unigram 250k (*WhitespaceSplit+Metaspace*, following XLM-R [5]), truncation 128 (*LongestFirst*), pad id 1. ONNX graph: inputs *input_ids*, *attention_mask*, *token_type_ids* [batch, seq]; output *last_hidden_state* [batch, seq, 384] (paraphrase-multilingual-MiniLM class [4], int8 ARM64, 113 MB). Pooling:

$$\mathbf{v} = \frac{\sum_i m_i \mathbf{h}_i}{\sum_i m_i}, \quad \hat{\mathbf{v}} = \mathbf{v} / \|\mathbf{v}\| \quad (1)$$

with mask m_i and hidden states $\mathbf{h}_i$. Query and document use the identical function (*embed_query* → *embed* when a model is present); on any ONNX/tokenizer error the chunk falls back to hash embeddings. Session access is guarded by *Mutex<Session>* because *Session::run* needs *&mut*.

2.2 Sparse path and fusion

Sparse TF-IDF over words seen at least twice (stored sparsely). Dense and sparse top-lists are fused:

$$\text{RRF}(d) = \frac{0.6}{60 + \text{rank}_{\text{dense}}(d)} + \frac{0.4}{60 + \text{rank}_{\text{sparse}}(d)} \quad (2)$$

Then: +0.3 financial boost (finance lexicon + more than 20 digits), +2.0 trigram-overlap on top-200 candidates (typo/morphology, cheap because not full-corpus), and rerank $0.5 \cdot \text{fusion} + 0.5 \cdot \text{cosine}(q_{\text{dense}}, d_{\text{dense}})$ reusing stored vectors (no re-embed).

3 CPU-Only Execution

ort 2.0.0-rc.13 with default CPU Execution Provider [11]; *tokenizers* 0.21; *ndarray* 0.17 for [1, seq] i64 tensors. No CUDA/CoreML, no Python at serving. Scoring (*cosine* dense + sparse intersect) uses *rayon par_iter* over 8 cores; PDF reading is thread-parallel; *mmap* handles files over 100 MB. Per-chunk inference is currently serial (*Mutex*) at ~27 ms/chunk — intra-op ORT threads still use multiple cores per *run*, but throughput is ~37 chunks/s. Batching 8–16 chunks per *run* and per-thread sessions remove this.

4 Benchmark Results

Machine: Apple M2, 8 cores, 16 GB, arm64, *cargo build --release*. Corpus *files/*: ANTAM FS 30 Jun 2026 (1.6 MB, 1909 chunks), Adaro (2.7 MB, 1612), 10840 (25 MB, 229). Total 3750 chunks. Queries: *tests/test_20.txt* (EN/ID/CN). Metric: keyword hit in top-10 snippet+expanded (BEIR-style hit-rate [12]; full MIRACL judgments [13] are future work).

4.1 Accuracy: 20/20

#	Query (short)	ms	Hit
1	total uang dari pelanggan	79.7	OK
2	pendapatan bersih ANTAM S1	66.6	OK
3	money from contracts (EN)	62.7	OK
4	selisih pendapatan–beban	65.4	OK
5	keuntungan sbkm operasi	63.3	OK
6	profit after tax	63.1	OK
7	nilai kekayaan perusahaan	53.3	OK
8	kewajiban harus dibayar	53.3	OK
9	bagian pemegang saham	50.8	OK
10	keuntungan dibagikan	48.3	OK
11	kas dari operasi	52.9	OK
12	jumlah emas diproduksi	48.1	OK
13	produksi nikel (ton)	45.4	OK
14	presiden direktur?	45.9	OK
15	produk tambang utama	46.9	OK
16	beban pajak penghasilan	47.2	OK
17	piutang pelanggan	46.4	OK
18	stok emas blm dijual	46.6	OK
19	selisih pendapatan–beban	46.8	OK
20	ANTAM 2026 H1 revenue (CN)	47.5	OK

Table 1: All 20 queries pass (100%), incl. Chinese Q20 that fails in hash-fallback mode.

4.2 Performance

Metric	Value	Target
Recall (20 queries)	100%	>95%
Avg query latency	54.0 ms	<100 ms
Min / max query	45.4 / 79.7 ms	<100 ms
Index (3750 chunks)	~100 s	<60 s
Throughput	~37 chunk/s	—
Embedding dim	384	—
GPU	none (CPU)	CPU-only

Table 2: Query path meets the <100 ms target; indexing exceeds 60 s (serial inference).

4.3 Scaling

Corpus	Chunks	Share
ANTAM (1.6 MB)	1909	51%
Adaro (2.7 MB)	1612	43%
10840 (25 MB)	229	6%

Corpus	Chunks	Share
Combined	3750	100%

Table 3: Chunk distribution. Index time is linear in chunk count.

5 Comparison with Baselines

System	Recall	Query	Index
Hash fallback	19/20	~2 ms	~2–8 s
onod+ORT CPU	20/20	~54 ms	~100 s
Dense MiniLM Py	~80%	~55 ms	~15 s*
SPLADE [8]	100%†	~1 ms	~4 s†

Table 4: *GPU-assisted; †stub on ANTAM. ORT wins recall, pays indexing.

Pure BM25 [1] matches keywords at ~1 ms but misses paraphrases; dense-only MiniLM [4] needs GPU for indexing and drops exact numbers; SPLADE [8] / ColBERTv2 [9] are strong but heavier to serve. onod keeps BM25-like exactness (sparse + number boost) and adds cross-lingual dense (BGE-M3 [6] / E5 [7] class ideas, distilled to a 384-dim MiniLM) without a vector DB or GPU.

6 Implementation Details

Rust *core/src/main.rs* (~790 lines): *TransformerEmbedder* (Mutex<Option<Session>> + Tokenizer), *SparseEmbedder*, *Reranker*, NFKC normalization, sentence-aware chunking, *OnodIndex* (bincode *index.bin*), CLI (*benchmark/search/serve*), stdlib REST server. Deps: *ort/tokenizers/ndarray/rayon/unicode-normalization/memmap2/serde/pdf_oxide*. Key correctness fixes vs the broken stub: load the real *tokenizer.json* (was empty WordPiece → MissingUnkToken panic), send all 3 inputs (was 2 → ORT error), mean-pool instead of raw *take(dim)*, and *embed_query* consistency (was hash-vs-model mismatch).

7 Limitations and Future Work

Limitations: (1) indexing ~100 s exceeds the 60 s target (serial Mutex inference); (2) trigram index memory; (3) keyword-hit metric, not full nDCG/MRR on MIRACL [13]/BEIR [12]; (4) single 128-token truncation per chunk may clip long tables.

Next: (1) batch inference 8–16 chunks/run; (2) per-thread sessions; (3) WAND/Block-Max WAND [15] pruning with top-200 to top-20 rerank; (4) persistent/incremental dense index; (5) eval on MIRACL/MrTyDi/BEIR-multilingual; (6) optional cross-encoder reranker [10] on candidates only.

8 Conclusion

onod shows a full-Rust, CPU-only hybrid — ONNX Transformer dense + sparse TF-IDF with RRF, boosts, and rerank — reaches 20/20 (100%) trilingual recall at 54 ms average query latency without a GPU. The remaining gap is CPU indexing throughput, addressable by batching and session parallelism while keeping the query path under 100 ms.

References

- [1] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Found. Trends Inf. Retr.*, 2009.
- [2] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *SIGIR*, 2009.
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *EMNLP*, 2019.
- [4] N. Reimers and I. Gurevych, “Making monolingual sentence embeddings multilingual using knowledge distillation,” 2020.
- [5] A. Conneau et al., “Unsupervised cross-lingual representation learning at scale,” in *ACL*, 2020.
- [6] J. Chen et al., “BGE-M3: Multi-lingual, multi-functionality, multi-granularity text embeddings,” 2024.
- [7] L. Wang et al., “Multilingual E5 text embeddings,” 2024.
- [8] T. Formal et al., “SPLADE v2: Sparse lexical and expansion model for information retrieval,” 2021.
- [9] K. Santhanam et al., “ColBERTv2: Effective and efficient retrieval via lightweight late interaction,” in *SIGIR*, 2022.
- [10] V. Karpukhin et al., “Dense passage retrieval for open-domain question answering,” in *EMNLP*, 2020.
- [11] ONNX Runtime Developers, “ONNX Runtime: CPU execution provider, quantization and performance,” 2024.
- [12] N. Thakur et al., “BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models,” in *NeurIPS Datasets*, 2021.
- [13] X. Zhang et al., “MIRACL: Multilingual information retrieval across a continuum of languages,” 2023.
- [14] J. Lin et al., “Pyserini: A Python toolkit for reproducible information retrieval research with sparse and dense representations,” in *SIGIR*, 2021.
- [15] A. Broder et al., “Efficient query evaluation using a two-level retrieval process,” in *CIKM*, 2003.

Appendix A Reproducibility

One-command CPU benchmark (Apple M2, 8-core):

```
cd core
# place model.onnx (113M qint8) + tokenizer.json
# (8.7M) in core/ (git-ignored)
cargo build --release
./target/release/onod benchmark ../files/
# expect: ORT model loaded, 3750 chunks, 20/20 =
100%, avg ~54ms
```

Sources: core/src/main.rs (ORT+hybrid), core/src/lib.rs (FFI BM25), tests/test_20.txt (20 queries), benchmarks/ANTAM_benchmark.md (Python baselines), files/ (3 PDFs). Model files (*.onnx, tokenizer.json) are git-ignored.